

Problem Description:

Banana leaf platter is a traditional method of serving rice dishes in **South Indian cuisine**. Due to the migration of South Indians, banana leaf rice can also be found in areas with significant ethnic South Indian diaspora such as **Malaysia** and **Singapore**.

Irfan is a banana leaf sales person.
he has N stacks of banana leafs.

Question No:9

Each stack contains K leafs.

Each leaf has a positive beauty value, describing how attractive it looks.

Irfan would like to take exactly P leafs to use for lunch today. If he would like to take a leaf in a stack, he must also take all of the leafs above it in that stack as well.

Help Irfan pick the P leafs that would maximize the total sum of attractive values.

Constraints:

$1 \leq T \leq 100.$

$1 \leq K \leq 30.$

$1 \leq P \leq N * K.$

$1 \leq N \leq 50.$

Input Format:

The first line of the input gives the number of test cases, T. T test cases follow. Each test case begins with a line containing the three integers N, K and

P. Then, N lines follow. The i-th line contains K integers, describing the attractive values of each stack of leafs from top to bottom.

Output Format:

Print the output in a separate line contains the maximum total sum of attractive values that Irfan could pick.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
2 4 5
10 10 100 30
80 50 10 50
3 2 3
80 80
20 20
15 50
```

EXPECTED OUTPUT

```
250
180
```

Test Case 2

INPUT (STDIN)

```
2
2 4 5
20 20 70 30
60 30 10 50
3 2 3
60 70
15 40
20 10
```

EXPECTED OUTPUT

```
200
150
```

▼ Mandatory Test Cases

Test Case 1

KEYWORD

```
int max(int a,int b)
```

Test Case 2

KEYWORD

```
for(int i = 0;i < n;i++)
```

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #define MAX 1000000000000000LL
4  int main() {
5      int T;
6      scanf("%d", &T);
7      while (T--) {
8          int N, K, X;
9          scanf("%d %d %d", &N, &K, &X);
10
11         long long dp[X + 1];
12
13         for (int i = 0; i <= X; i++) {
14             dp[i] = -MAX;
15         }
16
17         dp[0] = 0;
18
19         for (int i = 0; i < N; i++) {
20
21             long long prefix[K + 1];
22             prefix[0] = 0;
23
24             for (int j = 1; j <= K; j++) {
25                 long long value;
26                 scanf("%lld", &value);
27                 prefix[j] = prefix[j - 1] + value;
28             }
```

```

29
30     long long newdp[X + 1];
31
32     for (int j = 0; j <= X; j++) {
33         newdp[j] = -MAX;
34     }
35
36     for (int taken = 0; taken <= X; taken++) {
37
38         if (dp[taken] == -MAX)
39             continue;
40
41         for (int take = 0; take <= K && taken + take <= X; take++) {
42
43             long long value = dp[taken] + prefix[take];
44
45             if (value > newdp[taken + take]) {
46                 newdp[taken + take] = value;
47             }
48         }
49     }
50     for (int j = 0; j <= X; j++) {
51         dp[j] = newdp[j];
52     }
53 }
54 printf("%lld\n", dp[X]);
55
56     return 0;
57 }

```

Output:

Test Case-1:

	2	
	2 4 5	
	10 10 100 30	
	80 50 10 50	
	250	
	3 2 3	
	80 80	
	20 20	
	15 50	
	180	

Test Case-2:

	2	
	2 4 5	
	20 20 70 30	
	60 30 10 50	
	200	
	3 2 3	
	60 70	
	15 40	
	20 10	
	150	

Keshav Kumar Rauniyar
CH.SC.U4CSE25236